Predicting the Next Best Track in a DJ Set

Lavinia Elser[1] **Saurish Kapoor**[1]
Johnathan Rafailov[1] **Jasmine Hanjra**[1]
[1] Courant Institute of Mathematical Sciences
New York University
{le2396,sk12995,rafaij01,jh10489}@nyu.edu

Abstract

DJs spend countless hours combing through their music libraries to find songs that mix well together, taking time away from the actual craft of executing seamless transitions. We address this problem by training a recommendation system that takes a song's audio features as input and recommends the top 5 most mixable tracks. Unlike existing music recommenders that rely on user taste, genre, or artist metadata, we trained our models exclusively on the acoustic properties of audio. We framed the task as binary pair classification, with positive pairs drawn from real DJ transitions and negative pairs sampled from non-transitions. We compared six models including Random Forest, Gradient Boosting, LightGBM, PCA with Random Forest, and a Siamese contrastive network. LightGBM achieved the highest test accuracy at 78.6% under random negative sampling, with harmonic BPM distance and tempo emerging as the most predictive features. A survey of 35 participants (20% professional DJs) reported 67.84% overall satisfaction with the recommended tracks.

1 Introduction

In this project we carried out a supervised learning problem which addresses music matching for DJs. DJs spend a lot of time looking through their music library before picking the next song, especially if the library is disorganized. While, they could use their time perfecting the next transition. Since the pressure is high, we want to facilitate DJs in choosing their next song, in order for them to have more time to focus on the actual mix.

The model created acts a recommendation platform for DJs to facilitate them in choosing the next tracks to play based on some physical properties of the song derived from audio itself. Given a library of N tracks and a query song s the DJ is actively playing, the model ranks the remaining $N-1$ tracks by how mixable each one is with the query s, and return the top five by rank.

The system will extract features from the input song such as its tempo (beats per minute [BPM]), energy, and its rhythmic structure, in order to establish the ranking. Generally, a mix is well performed if the BPMs are similar (as excessively changing the BPM will disturb listeners), if their amplitude/energy match (coherency between the tracks), and if the rhythmic structures have similar patterns (to layer cleanly during the fade-in and fade-out of the two tracks).

The model created is different from previous music recommendation system as it does not base the choice of the song on the genre, artist nor user behavior, but rather uses interesting physical insights of the song to recommend others from a DJ prospective. The model was trained on real DJ mixes based on a pair classifier trained on observed adjacent track transitions. We defined positive and negative pairs respectively as two songs that could be mixable or not. Multiple methods were tested in order to find the most accurate and unbiased model.

Preprint.

A survey was sent out for human evaluation, and a retrieval of next tracks was done to establish the accuracy of our system. Findings indicate that gradient boosting was the best method to train the model with an accuracy of 77%. The most important feature that qualifies a positive pair was found to be tempo, which was expected as that is the first characteristic DJs look at when choosing songs.

Results from the survey show overall 67.84% satisfaction for the recommended tracks and coherent ranks, however with some room for improvement in the House genre. The track retrieval shows much better results fromthe applying gradient boosting than compared to the random baseline.

2 Related Work

When we started this project, we looked at what already existed for music recommendation and realized most of it was solving a slightly different problem than ours. Systems like the one studied in [Aljanaki et al., 2017] are built around emotional tags and listener behavior, they are trying to predict what a person might enjoy based on their profile history. That makes sense for a streaming platform, but a DJ needs more than that. The next song has to actually transition well into whatever is currently playing. This means that the tempo, energy, rhythm and overall song should fit well enough for the transition to feel natural.

There is also work on automatic DJ systems [Chen et al., 2021] that deals with transitions and beat matching, which is closer to what we are doing. But those systems just do everything on their own. That was not the point of our project. We were not trying to replace the DJ; we just wanted to make it easier to find the next song. The DJ still needs to be the one deciding, because a lot of DJing depends on the crowd and the moment, not just whether two songs technically work together.

Deej-AI [Teticio, 2021] was the closest thing to our project that we found. It listens to the audio instead of just looking at metadata, which is what we wanted to do too. But it was trained on Spotify playlists, so it is really optimized for what sounds good to a listener, not what would work in a DJ transition. Two songs can sound similar and still be a bad mix if the BPMs do not line up or the energy shift feels off.

For our project, we focused on mixability using audio features. We extracted features such as tempo, chroma, tempogram, MFCCs, RMS energy, and other rhythmic and sound texture features using librosa [McFee et al., 2015]. We also used harmonic BPM distance because DJs do not only mix songs with the exact same BPM. Sometimes songs can still work together at half-time or double-time relationships. Our project is not a music recommender and not an automatic DJ system either. It uses machine learning to help with track selection, while the DJ still makes the final call.

3 Method

3.1 Dataset

We use the open *mir-aidj djmix-dataset* [Kim et al., 2022] as our source of real DJ transitions. The dataset aggregates 5,040 DJ mixes from MixesDB which are annotated by the community, covering 63,038 unique tracks and 64,753 adjacent track transitions. Each mix entry has an ordered tracklist with track titles, positions in the mix, and a YouTube source URL for the mix audio. The dataset itself contains only the metadata; the audio for individual tracks have to be scraped separately.

Scraping audio. We first filtered the track set to those appearing in at least three distinct mixes so that the model is trained on tracks that DJs actually reach for repeatedly rather than one-off picks from a single mix. This collapses the long tail and brings the candidate pool down to roughly 4,100 tracks. For each candidate we then queried YouTube for the corresponding song and downloaded the audio with `yt-dlp` [yt-dlp contributors, 2024]. Around 10–20% of candidates failed to download for various reasons (404 error, age restrictions, etc.). After this stage we had 3,827 tracks with usable audio.

Cleaning the tracklist metadata. A non-trivial fraction of raw tracklist rows had titles that could not be parsed cleanly into an artist::title pair (formatting was inconsistent across mixes), and some unique track strings were near-duplicates of others (whitespace, "feat." versus "ft.", artist order). We deduped on a normalized version of the title and dropped rows whose track identifier could not

be matched back to a row in the audio features table. When an unparseable or unfeatured track sat between two clean tracks in a mix—tracklist [A, B, *C*, D]—we kept the surrounding pairs (A, B) and dropped (B, C) and (C, D) rather than adding a bridged pair (B, D); a positive is defined to be a real observed transition. We also dropped pairs where the same track appeared on both sides (a small number of self loops introduced by tracklist annotation errors).

Considerations during selection. Two tradeoffs drove the choices above. We preferred the djmix-dataset over scraping 1001Tracklists or building our own corpus because it is openly redistributable as metadata and the audio acquisition layer is intentionally separated from the tracklist information. Second, every cleaning step was set up so that it could only *remove* rows: we drop tracks rather than impute features, drop pairs rather than bridge missing tracks, and drop unmatched titles rather than match them. The resulting positives are conservative but each one corresponds to a transition a human DJ had actually played.

Final dataset. Of the 64,753 raw transitions, 3,724 had both endpoint tracks present in the audio features table; after dropping self-loops we retained 3,721 positive pairs spanning 1,605 distinct mixes. We sample negative pairs at a 1:1 ratio, giving a total of 7,442 labelled pairs. We split at the mix level, holding out 20% of contributing mixes for testing so that no mix appeared in both partitions. The resulting partition contained 5,940 training pairs and 1,502 test pairs.

3.2 Feature Extraction

We extracted acoustic features from every .mp3 file with the librosa Python package [McFee et al., 2015], summarizing each per-frame time series into per-track means and standard deviations. The raw extraction produced a 1,602-column feature table; we then restricted the modelling input to an allow-list of ten feature families so that highly correlated raw representations such as `melspectrogram` and `chroma_stft` did not dominate. The kept families cover three musical dimensions central to DJ practice: tempo (`tempo`, `tempogram_ratio`), energy and rhythmic texture (`rms`, `zero_crossing_rate`), and spectral / harmonic / timbral content (`spectral_centroid`, `spectral_bandwidth`, `spectral_contrast`, `spectral_flatness`, `chroma_cens`, `tonnetz`, `mfcc`). After filtering, each track is represented by a 113-dimensional feature vector.

DJs mix at three rhythmically compatible speeds: same tempo (1:1), double-tempo (2:1), and half-tempo (1:2). To capture this directly, we engineered a custom feature called *harmonic BPM distance*, defined as:

$$H_{BPM} = \min(|T_A - T_B|, |T_A - 2T_B|, |T_A - T_B/2|) \tag{1}$$

Given a song A with tempo T_A and a song B with tempo T_B, minimizing each term of eq. (1) leads to a perfect superposition of the two tracks under any of the three DJ-relevant ratios. The full feature extraction code is available at `https://github.com/jrafailov/musikal`.

3.3 Pair Construction and Negative Sampling

To finalize the construction of our dataset and make it training-ready, the next step was to determine the target variable. Since we wanted to recommend songs with high predicted mixability, we framed this recommendation task as a binary pair classification problem. We represent each pair as a feature vector $[A, B, |A-B|, H_{BPM}(A, B)]$, where A and B are the per-track librosa features. The absolute difference $|A - B|$ encodes how the two tracks differ across each feature, while H_{BPM} captures harmonic tempo compatibility between the two songs.

Positive pairs are drawn directly from observed adjacent transitions in the DJ Mix Dataset: if two songs were played one after the other in the same mix, they are considered a positive pair.

Negative pairs require a sampling strategy, since not all non-transitions are equally informative about whether two songs are incompatible. We evaluated two strategies:

- **Random negatives**: Two tracks sampled uniformly at random from the track pool, with the constraint that the pair is not a known positive transition in either direction.

- **Hard negatives**: For each positive anchor track, we sample a candidate within ± 5 BPM that is not a known transition. This forces the model to learn distinctions beyond simple BPM matching.

To assess the compatibility of each negative sampling strategy with our engineered harmonic BPM distance feature, we computed the mean and median harmonic distance across positive and negative pairs under each strategy. The results are shown in Figures 1 and 2.

```
============================================================
HARMONIC BPM DISTANCE STATISTICS
============================================================

Positive pairs (real DJ transitions):
  Mean harmonic distance: 4.90 BPM
  Median:                 2.45 BPM

Negative pairs (random):
  Mean harmonic distance: 11.36 BPM
  Median:                 5.81 BPM

Train: (5940, 4807), positives: 2974, negatives: 2966
Test:  (1502, 4807), positives: 747, negatives: 755
```

Figure 1: Harmonic BPM distance statistics under random negative sampling. Positive pairs have a mean harmonic distance of 4.90 BPM (median 2.45) compared to 11.36 BPM (median 5.81) for random negatives, providing clear separation between the two classes.

```
============================================================
HARMONIC BPM DISTANCE STATISTICS
============================================================

Positive pairs (real DJ transitions):
  Mean harmonic distance: 4.90 BPM
  Median:                 2.45 BPM

Negative pairs (BPM-matched hard negatives):
  Mean harmonic distance: 1.86 BPM
  Median:                 2.72 BPM

Train: (5940, 4807), positives: 2974, negatives: 2966
Test:  (1502, 4807), positives: 747, negatives: 755
```

Figure 2: Harmonic BPM distance statistics under hard negative sampling. The relationship inverts: hard negatives have a mean harmonic distance of 1.86 BPM (median 2.72), which is smaller than the 4.90 BPM mean of positive pairs. Because hard negatives are constrained within ± 5 BPM of a positive anchor, they end up closer in tempo than the positive pairs themselves.

Under random negative sampling, the harmonic BPM distance feature carries strong discriminative signal: positive pairs have substantially lower harmonic distance than negatives. Under hard negative sampling, this relationship inverts, with negatives showing a smaller mean harmonic distance than positives. This is a direct consequence of the hard sampling constraint, which by construction selects negatives that are BPM-similar to a positive anchor. Because harmonic BPM distance is a feature we engineered specifically to capture the tempo compatibility that DJs rely on when choosing tracks, we chose random negative sampling for our primary pipeline. Hard negatives remain valuable as a stricter evaluation strategy that removes the easy BPM cue, and we report results under both strategies in Section 4 to provide a complete picture of model performance.

3.4 Models

We trained six models on the binary pair classification task.

- **Random Forest (default)**: 100 trees, no maximum depth, default sklearn settings. This serves as our baseline. We chose Random Forest over logistic regression or other generalized linear models because we wanted to maintain a human in the loop and prioritize feature interpretability — Random Forest exposes per-feature importance, while linear models capture only linear relationships.
- **Random Forest (tuned)**: 500 trees, maximum depth 20, minimum 5 samples per split. We added regularization through capped tree depth to prevent overfitting, since the default model showed perfect train accuracy. We selected this hyperparameter combination through a grid search over discrete values, choosing the configuration that yielded the highest test accuracy.
- **Gradient Boosting**: 100 trees built sequentially with learning rate 0.1 and maximum depth 3. We chose another tree-based model to preserve interpretability, but with a different ensemble strategy: each new tree corrects errors from the previous trees, which often outperforms parallel ensembles like Random Forest on tabular data.
- **LightGBM**: 200 trees, learning rate 0.1, 31 leaves per tree. A faster gradient boosting variant that uses histogram-based split finding rather than the exact split-finding used in standard gradient boosting. We chose LightGBM because it typically outperforms standard gradient boosting on tabular data while training significantly faster — useful for hyperparameter exploration on our limited compute budget.
- **PCA + Random Forest**: We first reduced the per-track feature space to 10 principal components, then trained a Random Forest on the reduced representation. The harmonic BPM distance was added as a separate feature after PCA. We applied this approach to test whether the high dimensionality of our feature set was contributing noise rather than signal. PCA is restricted to linear combinations of the original features, so it captures redundancy in the linear structure of the data.
- **Siamese Contrastive Network**: A 3-layer fully-connected tower (input $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 16) with dropout 0.5, trained with contrastive loss (margin 1.0). Both tracks pass through the same shared network, producing 16-dimensional embeddings. Contrastive loss pulls mixable pairs close together in embedding space and pushes non-mixable pairs apart. We trained for up to 100 epochs with early stopping (patience 5) to prevent overfitting on our small dataset. We chose a Siamese architecture because the task involves comparing pairs of inputs: passing both tracks through the same shared network ensures the comparison is symmetric and prevents the model from learning different rules for each track. We wanted to validate whether there were hidden nonlinear structures in the acoustic features that linear or tree-based models could not capture.

For tree-based models, we report feature importance using sklearn's `feature_importances_` attribute. For PCA + Random Forest, we additionally report which original features contribute most to each principal component, allowing interpretation of the latent dimensions. The Siamese network learns embeddings end-to-end and does not directly expose per-feature importance. All models use a fixed random seed (42) to ensure reproducibility.

This section should describe your solution in detail. Clearly explain how your method works, including any models, algorithms, or design choices. Highlight how your solution is that different from others' solutions.

4 Experiments

[...] Moreover, in order to have a human evaluation, a survey was sent out consisting of five queries each one ranking the five most mixable songs. Users gave scores from 0 to 5, 0 indicating no compatibility and 5 indicating perfect mixability. The survey was completed by 35 people, 20% of whom are professional DJs, and some of whom are from the SoundCloud team. The five queries in the survey were all very different to each others, as can be seen from Table 1, in order to evaluate the results of our model across multiple genres.

Table 1: Illustrating the genres of the songs sent out in the survey, indicating a diversification between the tracks.

Title - Artist	Genre
Elsa - Johann Stone	Progressive Trance
Twelve - Psyk Rework	Tech House
Peace Keeper - Moving Fusion	Drum & Bass
No Distance - Dixon, Guy Gerber	Progressive House
Music Sounds Better With You - Stardust	Funky House

Results from the survey show an overall satisfaction of the model of 67.84%. As it can be seen in Figure 3, the song which received the highest score is *Twelve - Psyk Rework*, which is coherent as from Table 1 we can see that its genre is Tech House and the dataset used to train the model is mainly Tech House dominated. On the other hand, the song with the lowest score is *Music Sounds Better With You - Stardust* which is a Funky House song and our dataset did not have any Funky House, therefore the prediction is less accurate. The remaining tree songs in Figure 3 have somewhat similar scores oscillating between 3.25 and 3.54, as in fact, their genres represent in between 6.9% and 18.7% of the dataset. It can be notice that the score of a song increases as the percentage of its genre in the dataset increases, which is expected as it means the model will find more similar songs to the query.

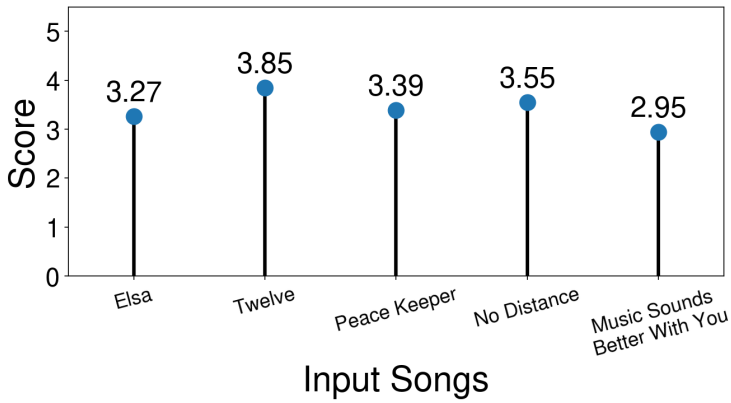

Figure 3: Mean score out of 5 received for the outputs coming from the 5 queries listed in Table 1 after sending out the survey. The overall satisfaction is of 67.84%.

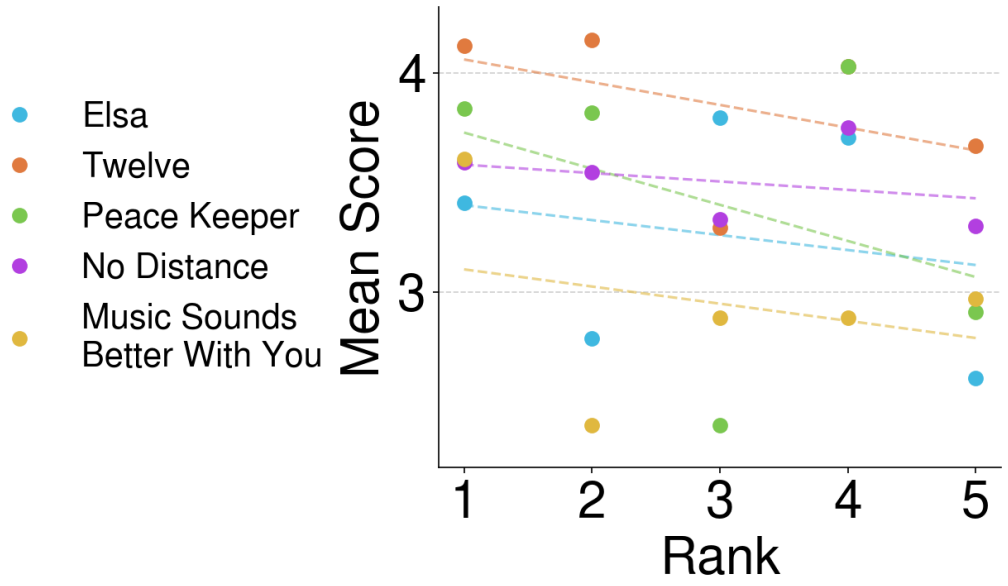

Figure 4: Illustrating the mean score per output per query from the survey, plotted against its rank given from the model with a best fit line. Showing that as the rank increases the mean score decreases for all queries.

In Figure 4, the data points collected from the survey are somewhat sparse, therefore a best-fit line was fitted to all of the songs scores to see whether the rank actually impacted the scores. Results show that all the best-fit lines are decreasing, which means that the rank position of the songs is somewhat accurate: the better the output song is ranked by the model, the more it is mixable with the query. As expected *Twelve - Psyk Rework* is the song with the highest mean score best-fit line, while *Music Sounds Better With You - Stardust* is the one with the lowest best-fit line, because of the dataset genre distribution.

This section should describe how you evaluate your method. Specify datasets, baselines, and evaluation metrics. Clearly describe which experiments you run. For each experiment, state what you expect to observe and justify why your method should perform better than existing approaches. Present results clearly and interpret them.

5 Discussion

[...]

The dataset used to train the model is mainly Tech House dominated, explaining why Funky House songs received worst scores than other tracks. This genre bias could be reduced if the dataset covered more songs and genres.

Moreover, a future investigation could be carried out to impose the directionality of the mixes: song A mixes with song B, if they are consecutive in the table, but the opposite is not verified. This allows the model to take into count the intro/outro structure of the songs, which at the moment is ignored given the way the positive pairs are defined.

Another consideration made for future work was finding more realistic hard negative pairs without introducing bias, which could elevate the accuracy of the model.

This section should summarize your findings and reflect on their implications. Discuss limitations of your approach and possible reasons for unexpected results. Suggest directions for future work.

References

[1] A. Aljanaki, Y.-H. Yang, and M. Soleymani. Developing a benchmark for emotional analysis of music. *PLOS ONE*, 12(3):e0173392, 2017.

[2] B.-Y. Chen, W.-H. Wang, and Y.-H. Yang. Automatic dj transitions with differentiable audio effects and generative adversarial networks. *arXiv preprint arXiv:2110.06525*, 2021.

[3] T. Kim, Y.-H. Yang, and J. Nam. Joint estimation of fader and equalizer gains of dj mixers using convex optimization. In *International Conference on Digital Audio Effects (DAFx)*, pages 312–319, 2022.

[4] B. McFee, C. Raffel, D. Liang, D. P. Ellis, M. McVicar, J. Battenberg, and O. Nieto. librosa: Audio and music signal analysis in python. In *Proceedings of the 14th Python in Science Conference*, pages 18–24. SciPy, 2015.

[5] Teticio. Deej-ai: Create automatic playlists by using deep learning to listen to the music. `https://github.com/teticio/Deej-AI`, 2021.

[6] yt-dlp contributors. yt-dlp: A youtube-dl fork with additional features and fixes. `https://github.com/yt-dlp/yt-dlp`, 2024.

A Technical appendices and supplementary material

Additional results, figures, graphs, and proofs may be submitted with the paper submission. You can include a link to code repository, but do not upload a separate PDF file for the appendix. There is no page limit for the appendices.

Note: Anything beyond the first 6 pages of the main paper is optional reading for the graders. The paper should be able to stand alone without the appendix; for example, adding critical experiments that support the main claims to an appendix is inappropriate.

Additional Content to Sprinkle Throughout: